

Troubleshoot the Reduce Function

NOTE

Aggregation Pipeline as Alternative to Map-Reduce

Starting in MongoDB 5.0, map-reduce is deprecated:

- Instead of map-reduce, you should use an aggregation pipeline. Aggregation pipelines provide better performance and usability than map-reduce.
- You can rewrite map-reduce operations using aggregation pipeline stages, such as `$group`, `$merge`, and others.
- For map-reduce operations that require custom functionality, you can use the `$accumulator` and `$function` aggregation operators. You can use those operators to define custom aggregation expressions in JavaScript.

For examples of aggregation pipeline alternatives to map-reduce, see:

- [Map-Reduce to Aggregation Pipeline](#)
- [Map-Reduce Examples](#)

An aggregation pipeline is also easier to troubleshoot than a map-reduce operation.

The `reduce` function is a JavaScript function that "reduces" to a single object all the values associated with a particular key during a map-reduce operation. The `reduce` function must meet various requirements. This tutorial helps verify that the `reduce` function meets the following criteria:

- The `reduce` function must return an object whose *type* must be **identical** to the type of the `value` emitted by the `map` function.
- The order of the elements in the `valuesArray` should not affect the output of the `reduce` function.
- The `reduce` function must be *idempotent*.

For a list of all the requirements for the `reduce` function, see `mapReduce`, or `mongosh` helper method `db.collection.mapReduce()`.

Confirm Output Type

You can test that the `reduce` function returns a value that is the same type as the value emitted from the `map` function.

1. Define a `reduceFunction1` function that takes the arguments `keyCustId` and `valuesPrices`. `valuesPrices` is an array of integers:

```
var reduceFunction1 = function(keyCustId, valuesPrices) {  
  return Array.sum(valuesPrices);  
}
```



```
};
```

2. Define a sample array of integers:

```
var myTestValues = [ 5, 5, 10 ];
```

3. Invoke the `reduceFunction1` with `myTestValues`:

```
reduceFunction1('myKey', myTestValues);
```

4. Verify the `reduceFunction1` returned an integer:

```
20
```

5. Define a `reduceFunction2` function that takes the arguments `keySKU` and `valuesCountObjects`. `valuesCountObjects` is an array of documents that contain two fields `count` and `qty`:

```
var reduceFunction2 = function(keySKU, valuesCountObjects) {  
  reducedValue = { count: 0, qty: 0 };  
  for (var idx = 0; idx < valuesCountObjects.length; idx++) {  
    reducedValue.count += valuesCountObjects[idx].count;  
    reducedValue.qty += valuesCountObjects[idx].qty;  
  }  
  return reducedValue;  
};
```

6. Define a sample array of documents:

```
var myTestObjects = [  
  { count: 1, qty: 5 },  
  { count: 2, qty: 10 },  
  { count: 3, qty: 15 }  
];
```

7. Invoke the `reduceFunction2` with `myTestObjects`:

```
reduceFunction2('myKey', myTestObjects);
```

8. Verify the `reduceFunction2` returned a document with exactly the `count` and the `qty` field:

```
{ "count" : 6, "qty" : 30 }
```

Ensure Insensitivity to the Order of Mapped Values

The `reduce` function takes a `key` and a `values` array as its argument. You can test that the result of the `reduce` function does not depend on the order of the elements in the `values` array.

1. Define a sample `values1` array and a sample `values2` array that only differ in the order of the array elements:

```
var values1 = [
  { count: 1, qty: 5 },
  { count: 2, qty: 10 },
  { count: 3, qty: 15 }
];
var values2 = [
  { count: 3, qty: 15 },
  { count: 1, qty: 5 },
  { count: 2, qty: 10 }
];
```

2. Define a `reduceFunction2` function that takes the arguments `keySKU` and `valuesCountObjects`. `valuesCountObjects` is an array of documents that contain two fields `count` and `qty`:

```
var reduceFunction2 = function(keySKU, valuesCountObjects) {
  reducedValue = { count: 0, qty: 0 };
  for (var idx = 0; idx < valuesCountObjects.length; idx++) {
    reducedValue.count += valuesCountObjects[idx].count;
    reducedValue.qty += valuesCountObjects[idx].qty;
  }
  return reducedValue;
};
```

3. Invoke the `reduceFunction2` first with `values1` and then with `values2`:

```
reduceFunction2('myKey', values1);
reduceFunction2('myKey', values2);
```

4. Verify the `reduceFunction2` returned the same result:

```
{ "count" : 6, "qty" : 30 }
```

Ensure Reduce Function Idempotence

Because the map-reduce operation may call a `reduce` multiple times for the same key, and won't call a `reduce` for single instances of a key in the working set, the `reduce` function must return a value of the same type as the value emitted from the `map` function. You can test that the `reduce` function process "reduced" values without affecting the *final* value.

1. Define a `reduceFunction2` function that takes the arguments `keySKU` and `valuesCountObjects`. `valuesCountObjects` is an array of documents that contain two fields `count` and `qty`:

```
var reduceFunction2 = function(keySKU, valuesCountObjects) {  
  reducedValue = { count: 0, qty: 0 };  
  for (var idx = 0; idx < valuesCountObjects.length; idx++) {  
    reducedValue.count += valuesCountObjects[idx].count;  
    reducedValue.qty += valuesCountObjects[idx].qty;  
  }  
  return reducedValue;  
};
```

2. Define a sample key:

```
var myKey = 'myKey';
```

3. Define a sample `valuesIdempotent` array that contains an element that is a call to the `reduceFunction2` function:

```
var valuesIdempotent = [  
  { count: 1, qty: 5 },  
  { count: 2, qty: 10 },  
  reduceFunction2(myKey, [ { count: 3, qty: 15 } ] )  
];
```

4. Define a sample `values1` array that combines the values passed to `reduceFunction2`:

```
var values1 = [  
  { count: 1, qty: 5 },  
  { count: 2, qty: 10 },  
  { count: 3, qty: 15 }  
];
```

5. Invoke the `reduceFunction2` first with `myKey` and `valuesIdempotent` and then with `myKey` and `values1`:

```
reduceFunction2(myKey, valuesIdempotent);  
reduceFunction2(myKey, values1);
```

6. Verify the `reduceFunction2` returned the same result:

```
{ "count" : 6, "qty" : 30 }
```